

05-2 파이썬 [모듈]

모듈로 만들어야 실행 가능

관련 기능을 하는

확장자가 py인 파일

모듈이란 함수나 변수 또는 클래스 등을 모아 놓은 파일이다. 모듈은 다른 파이썬 프로그램에서 불러와 사용할 수 있게끔 만들어진 파이썬 파일이라고도 할 수 있다. 우리는 파이썬으로 프로그래밍을 할 때 굉장히 많은 모듈을 사용한다. 다른 사람들이 이미 만들어 놓은 모듈을 사용할 수도 있고 우리가 직접 만들어서 사용할 수도 있다. 여기서는 모듈을 어떻게 만들고 사용할 수 있는지 알아보겠다.

모듈 만들고 불러 보기

큰 단위
(하나 파일 안에 여러 기능)



App
App(Module)
App(Module)
App(Module) 작은 단위

큰 코드 단위 개발

- 문제점
 1. 어디에 무슨 코드 있는지 찾기 어렵.
 2. 중복 코드 다수 가능성(다른 곳에서 쓰려면 복붙해야.)
 3. 협업 시 문제(충돌)
- 작은 코드 단위 개발(모듈 단위)
 1. 작은 단위 카테고리화하여 찾기 쉬움
 2. 다른 모듈에서 import 가능
 3. 다른 모듈 수정으로 협업 문제 해결

모듈명.확장자
mod1.py

```
def sum(a, b):  
    return a + b
```

위와 같이 sum 함수만 있는 파일 mod1.py를 만들고 C:\Python 디렉터리에 저장하자. 이 파일이 바로 모듈이다. 지금까지 에디터로 만들어 왔던 파일과 다르지 않다.

우리가 만든 mod1.py라는 파일, 즉 모듈을 파이썬에서 불러와 사용하려면 어떻게 해야 할까? 먼저 아래와 같이 도스 창을 열고 mod1.py를 저장한 디렉터리(이 책에서는 C:\Python)로 이동한 다음 **대화형 인터프리터**를 실행한다.
(=실행기)

```
C:\Users\Wpahkey>cd C:\Python  
C:\Python>dir 현재 디렉터리 안에 무엇이 있는지  
                확인할 때 사용하는 윈도우 명령어  
...             ls (mac/linux)  
2014-09-23 오후 01:53 49 mod1.py  
...
```

```
C:\Python>python  
Python 3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:54:25) [MSC v.1900 64 bit (AM...  
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```

반드시 mod1.py를 저장한 위치로 이동한 다음 이후 예제를 진행해야 한다. 그래야만 대화형 인터프리터에서 mod1.py를 읽을 수 있다. 이제 아래와 같이 따라 해보자.

앞에 아무것도 없이 mod1이니가 현재 디렉터리에 mod1.py 있다는 것.

```
>>> import mod1  
>>> print(mod1.sum(3,4))  
7
```

→ 인터프리터가 코드 실행
←

import 방법
1. import mod1 → sum() 활용
2. from mod1 import sum → mod1.sum(1,2)
3. import mod1 as m → sum(1,2)
→ m.sum(1,2)

from mod1 import * <- 절대 사용x

mod1.py를 불러오기 위해 import mod1이라고 입력하였다. import mod1.py로 입력하는 실수를 하지 않도록 주의하자. import는 이미 만들어진 파이썬 모듈을 사용할 수 있게 해주는 명령어이다. mod1.py 파일에 있는 sum 함수를 이용하기 위해서는 위의 예제와 같이 mod1.sum처럼 모듈이름

뒤에 '!(도트 연산자)를 붙이고 함수 이름을 써서 사용할 수 있다.

(※ import는 현재 디렉터리에 있는 파일이나 파이썬 라이브러리가 저장된 디렉터리에 있는 모듈만 불러올 수 있다.)

import의 사용 방법은 다음과 같다.

import 모듈이름

여기서 모듈이름은 mod1.py에서 .py라는 확장자를 제거한 mod1만을 가리킨다.

이번에는 mod1.py 파일에 다음 함수를 추가해 보자.

```
def safe_sum(a, b):
    if type(a) != type(b):
        print("더할 수 있는 것이 아닙니다.")
        return
    else:
        result = sum(a, b)
    return result
```

safe_sum 함수는 서로 다른 타입의 객체끼리 더하는 것을 미리 막아 준다. 만약 서로 다른 형태의 객체가 입력으로 들어오면 "더할 수 있는 값이 아닙니다"라는 메시지를 출력한다. 그리고 return문만 단독으로 사용되어 None 값을 돌려주고 함수를 종료한다.

이 함수를 mod1.py에 추가한 다음 다시 대화형 인터프리터를 열고 다음과 같이 따라 해보자.

```
>>> import mod1
>>> print(mod1.safe_sum(3, 4))
7
```

import mod1으로 mod1.py 파일을 불러온 다음 mod1.safe_sum(3, 4)로 safe_sum 함수를 호출한다. 이렇게 하면 같은 타입의 객체가 입력으로 들어와서 3+4의 결과인 7이 출력된다.

이번에는 다음처럼 따라 해보자.

```
>>> print(mod1.safe_sum(1, 'a'))
더할 수 있는 값이 아닙니다.
None
>>>
```

위 예제에서 1은 정수형 객체, a는 문자열 객체이다. 이렇게 서로 타입이 다른 객체가 입력으로 들어오면 "더할 수 있는 값이 아닙니다."라는 메시지를 출력하고 단독으로 사용된 return에 의해서 None 값을 돌려주게 된다.

mod1의 sum 함수 역시 다음처럼 바로 호출할 수도 있다.

```
>>> print(mod1.sum(10, 20))
30
```

[모듈 함수를 사용하는 또 다른 방법] 모듈 안에서 test 진행 후 이상 없으면 import

때로는 mod1.sum, mod1.safe_sum처럼 쓰지 않고 그냥 sum, safe_sum처럼 함수를 쓰고 싶은 경우도 있을 것이다. 이럴 때는 "from 모듈이름 import 모듈함수"를 사용하면 된다.

```
from 모듈이름 import 모듈함수
```

from ~ import ~를 이용하면 위와 같이 모듈이름을 붙이지 않고 바로 해당 모듈의 함수를 쓸 수 있다. 다음과 같이 따라 해보자.

```
>>> from mod1 import sum
>>> sum(3, 4)
7
```

그런데 위와 같이 하면 mod1.py 파일의 sum 함수만 사용할 수 있다. sum 함수와 safe_sum 함수를 둘 다 사용하고 싶다면 어떻게 해야 할까?

2가지 방법이 있다.

```
from mod1 import sum, safe_sum
```

첫 번째 방법은 위와 같이 from 모듈이름 import 모듈함수1, 모듈함수2처럼 사용하는 방법이다. 콤마로 구분하여 필요한 함수를 불러올 수 있다.

```
from mod1 import *
```

두 번째 방법은 위와 같이 * 문자를 사용하는 방법이다. 07장에서 배울 정규 표현식에서 * 문자는 "모든것"이라는 뜻인데 파이썬에서도 마찬가지로 의미로 사용된다. 따라서 from mod1 import *는 mod1.py의 모든 함수를 불러서 사용하겠다는 말이다.

mod1.py 파일에는 함수가 2개밖에 없기 때문에 위의 2가지 방법은 동일하게 적용된다.

if __name__ == "__main__": 의 의미 문제성 -> 해결방법 제시

이번에는 mod1.py 파일에 다음과 같이 추가해 보자.

```
# mod1.py
def sum(a, b):
    return a+b

def safe_sum(a, b):
    if type(a) != type(b):
```

```
print("더할수 있는 것이 아닙니다.")
```

```
return
```

```
else:
```

```
result = sum(a, b)
```

```
return result
```

test code

```
print(safe_sum('a', 1))
```

다른 모듈에서 import 했을 때 test code까지 나옴

```
print(safe_sum(1, 4))
```

```
print(sum(10, 10.4))
```

위와 같은 mod1.py 파일을 에디터로 작성해서 C:\Python이라는 디렉터리에 저장했다면 다음처럼 실행할 수 있다.

mod1.py 직접 실행하면 python interpreter

```
C:\Python>python mod1.py
```

```
더할 수 있는 것이 아닙니다.
```

```
None
```

```
5
```

```
20.4
```

```
__name__ = "__main__"
```

import mod1을 통해 실행하면
__name__="mod1"

mod1.sum() 이런 식으로 호출
하게 됨.

직접 실행할 때만 test code 실행되도록

결과값은 위의 예처럼 출력될 것이다. 그런데 이 mod1.py 파일의 sum과 safe_sum 함수를 사용하기 위해 mod1.py 파일을 import하면 문제가 생긴다.

도스 창을 열고 다음을 따라 해보자.

```
C:\WINDOWS> cd C:\Python
```

```
C:\Python>python
```

```
>>> import mod1
```

```
더할 수 있는 것이 아닙니다.
```

```
None
```

```
5
```

```
20.4
```

import 찾기 리스트 -> append

임시메모리

1. cache: sys.modules에 mod1 찾기.

True

(캐시된 모듈)

mod1 사용

False

추가

2. Built-in : 내장

mod1 사용

False

3. sys.path [현재 디렉토리, .., ...] append 추가

False

ModuleNotFoundError

text

mod1.py(현재 실행가능한 상태x)

영뚱하게도 import mod1을 수행하는 순간 mod1.py가 실행이 되어 결과값을 출력한다. 우리는 단지 mod1.py 파일의 sum과 safe_sum 함수만 사용하려고 했는데 말이다. 이러한 문제를 방지하려면 다음처럼 하면 된다.

Compile

bytecode

mod1.pyc

__pycache__에 저장

컴파일된 pyc를 저장하므로 다음 import할 때는 컴파일 다시 안 함.(시간 단축)

```
if __name__ == "__main__":
```

```
print(safe_sum('a', 1))
```

```
print(safe_sum(1, 4))
```

```
print(sum(10, 10.4))
```

if __name__ == "__main__"을 사용하면 C:\Python>python mod1.py처럼 직접 이 파일을 실행시켰을 때는 __name__ == "__main__"이 참이 되어 if문 다음 문장들이 수행된다. 반대로 대화형 인터프리터나 다른 파일에서 이 모듈을 불러서 사용할 때는 __name__ == "__main__"이 거짓이 되어 if문

다음 문장들이 수행되지 않는다.

파이썬 모듈을 만든 다음 그 모듈을 테스트하기 위해 보통 위와 같은 방법을 사용하는데, 실제로 그런지 대화형 인터프리터를 열고 실행해 보자.

```
>>> import mod1
>>>
```

mod1.py 파일의 마지막 부분을 위와 같이 고친 다음에는 아무런 결과값도 출력되지 않는 것을 볼 수 있다.

알아두기

파이썬의 `__name__` 변수는 파이썬이 내부적으로 사용하는 특별한 변수명이다. 만약 `C:\Python> python mod1.py`처럼 직접 mod1.py 파일을 실행시킬 경우 mod1.py의 `__name__` 변수에는 `__main__` 이라는 값이 저장된다. 하지만 파이썬 셸이나 다른 파이썬 모듈에서 mod1을 import 할 경우에는 mod1.py의 `__name__` 변수에는 "mod1"이라는 mod1.py의 모듈이름 값이 저장된다.

클래스나 변수 등을 포함한 모듈

지금까지 살펴본 모듈은 함수만 포함했지만 클래스나 변수 등을 포함할 수도 있다. 다음의 프로그램을 작성해 보자.

상수로 사용

↑ # mod2.py 모듈
변수 → `PI = 3.141592`

클래스 { `class Math:`
 `def solv(self, r):` 메소드
 `return PI * (r ** 2)` 바깥에 있는 변수 쓰면 유지보수 어렵.

함수 { `def sum(a, b):`
 `return a+b`

`if __name__ == "__main__":` mod2.py로 직접 실행하면
 `print(PI)`
 `a = Math()`
 `print(a.solv(2))`
 `print(sum(PI , 4.4))`

이 파일은 원의 넓이를 계산하는 Math 클래스와 두 값을 더하는 sum 함수 그리고 원주율 값에 해당되는 PI 변수처럼 클래스, 함수, 변수 등을 모두 포함하고 있다. 파일 이름을 mod2.py로 하고 `C:\Python` 디렉터리에 저장하자. mod2.py 파일은 다음과 같이 실행할 수 있다.

```
C:\Python>python mod2.py
```

```
3.141592
12.566368
7.541592
```

이번에는 대화형 인터프리터를 열고 다음과 같이 따라 해보자.

```
C:\Python>python
>>> import mod2
>>>
```

`__name__ == "__main__"`이 거짓이 되므로 아무런 값도 출력되지 않는다.

모듈에 포함된 변수, 클래스, 함수 사용하기

```
>>> print(mod2.PI)
3.141592
```

위의 예에서 볼 수 있듯이 `mod2.PI`처럼 입력해서 `mod2.py` 파일에 있는 `PI`라는 변수값을 사용할 수 있다.

```
>>> a = mod2.Math()
>>> print(a.solve(2))
12.566368
```

위의 예는 `mod2.py`에 있는 `Math` 클래스를 사용하는 방법을 보여 준다. 위의 예처럼 모듈 내에 있는 클래스를 이용하려면 `'.'`(도트 연산자)를 이용하여 클래스 이름 앞에 모듈 이름을 먼저 입력해야 한다.

```
>>> print(mod2.sum(mod2.PI, 4.4))
7.541592
```

`mod2.py`에 있는 `sum` 함수 역시 당연히 사용할 수 있다.

modtest.py mod2.py 새 파일 안에서 이전에 만든 모듈 불러오기

지금까지는 만들어 놓은 모듈 파일을 사용하기 위해 대화형 인터프리터만을 이용했다. 이번에는 새롭게 만들 파이썬 파일 안에 이전에 만들어 놓았던 모듈을 불러와서 사용하는 방법에 대해 알아보자.

방금 전에 만든 모듈인 `mod2.py` 파일을 새롭게 만들 파이썬 프로그램 파일에서 불러와 사용해 보자. 그럼 에디터로 다음과 같이 작성해 보자.

```
# modtest.py
import mod2
```

```
result = mod2.sum(3, 4)
print(result)
```

위에서 볼 수 있듯이 파일에서도 import mod2로 mod2 모듈을 불러와서 사용하면 된다. 대화형 인터프리터에서 한 것과 마찬가지로 방법이다. 위의 예제가 정상적으로 실행되기 위해서는 modtest.py 파일과 mod2.py 파일이 동일한 디렉터리에 있어야 한다.

[모듈을 불러오는 또 다른 방법]

cd ./.....

우리는 지금껏 도스 창을 열고 **모듈이 있는 디렉터리로 이동한 다음에나 모듈을 사용할 수 있었다.** 이번에는 모듈을 저장한 디렉터리로 이동하지 않고 모듈을 불러와서 사용하는 방법에 대해서 알아보자. 어디에서나

우선 이전에 만든 mod2.py 모듈을 C:\Python\Mymodules라는 디렉터를 새로 생성해서 저장한 후 다음의 예를 따라 해보자.

1. sys.path.append(모듈을 저장한 디렉터리) 사용하기

먼저 sys 모듈을 불러온다.

```
>>> import sys
```

sys 모듈은 파이썬을 설치할 때 함께 설치되는 라이브러리 모듈이다. sys에 대해서는 뒤에서 다시 다룰 것이다. 이 sys 모듈을 이용해서 파이썬 라이브러리가 설치되어 있는 디렉터를 확인할 수 있다.

다음과 같이 작성해 보자.

```
>>> sys.path
['', 'C:\\Windows\\SYSTEM32\\python35.zip', 'c:\\Python35\\DLLs',
'c:\\Python35\\lib', 'c:\\Python35', 'c:\\Python35\\lib\\site-packages']
```

sys.path는 파이썬 라이브러리들이 설치되어 있는 디렉터리들을 보여 준다. 만약 파이썬 모듈이 위의 디렉터리에 들어 있다면 모듈이 저장된 디렉터리로 이동할 필요없이 바로 불러서 사용할 수가 있다. 그렇다면 sys.path에 C:\Python\Mymodules라는 디렉터를 추가하면 아무데서나 불러 사용할 수 있지 않을까?

(※ 도스 창에서는 /, \든 상관없지만, 소스코드 안에서는 반드시 / 또는 \ 기호를 사용해야 한다.)

당연하다. sys.path의 결과값이 리스트이므로 우리는 다음과 같이 할 수 있을 것이다.

```
>>> sys.path.append("C:/Python/Mymodules") 1회성
>>> sys.path
['', 'C:\\Windows\\SYSTEM32\\python35.zip', 'c:\\Python35\\DLLs',
'c:\\Python35\\lib', 'c:\\Python35', 'c:\\Python35\\lib\\site-packages',
'C:/Python/Mymodules']
```

```
'C:/Python/Mymodules']
```

```
>>>
```

sys.path.append를 이용해서 C:/Python/Mymodules라는 디렉토리를 sys.path에 추가한 후 다시 sys.path를 보면 가장 마지막 요소에 C:/Python/Mymodules라고 추가된 것을 확인할 수 있다.

자, 실제로 모듈을 불러와서 사용할 수 있는지 확인해 보자.

```
>>> import mod2
```

```
>>> print(mod2.sum(3,4))
```

```
7
```

이상 없이 불러와서 사용할 수 있다. 이렇게 특정한 디렉토리에 있는 모듈을 불러와서 사용하고 싶을 때 사용할 수 있는 것이 바로 sys.path.append(모듈을 저장한 디렉토리)이다.

mymodule 등록 운영체제가 관리하는 변수 (컴퓨터 꺼지기)
sys.path, pythonpath 둘 다 잘 안 씀 운영체제가 종료하기 전에는 언제든지 사용 가능
패키지 많이 씀.
2. PYTHONPATH 환경 변수 사용하기
모듈을 불러와서 사용하는 또 다른 방법으로는 PYTHONPATH 환경 변수를 사용하는 방법이 있다.

다음과 같이 따라 해보자.

```
C:\Users\Whome> set PYTHONPATH=C:\Python\Mymodules
```

```
C:\Users\Whome> python
```

```
Python 3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:54:25) [MSC v.1900 64 bit (AM...
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

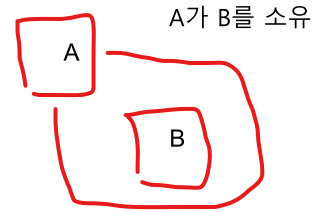
```
>>> import mod2
```

```
>>> print(mod2.sum(3,4))
```

```
7
```

set 도스 명령어를 이용해 PYTHONPATH 환경 변수에 mod2.py 파일이 있는 C:\Python\Mymodules 디렉토리를 설정한다. 그러면 디렉토리 이동이나 별도의 모듈 추가 작업 없이 mod2 모듈을 불러와서 사용할 수 있다.

패키지 모듈
A.B



05-3 파이썬 [패키지] 관련 모듈들을 모아 관리하는 방식

디렉토리 ← 파일

패키지(Packages)는 도트(.)를 이용하여 파이썬 모듈을 계층적(디렉터리 구조)으로 관리할 수 있게 해준다. 예를 들어 모듈명이 A.B인 경우 A는 패키지명이 되고 B는 A 패키지의 B 모듈이 된다.

파이썬 패키지는 디렉터리와 파이썬 모듈로 이루어지며 구조는 다음과 같다.

가상의 game 패키지 예

```
game/
├── __init__.py
├── sound/
│   ├── __init__.py
│   ├── echo.py
│   └── wav.py
├── graphic/
│   ├── __init__.py
│   ├── screen.py
│   └── render.py
└── play/
    ├── __init__.py
    ├── run.py
    └── test.py
```

각각의 디렉터리에 __init__이 있었기 때문에 패키지로 인식함.
game.sound.echo.py하려면 game, sound에 __init__이 있어야 함.

game, sound, graphic, play는 디렉터리명이고 .py 확장자를 가지는 파일은 파이썬 모듈이다. game 디렉터리가 이 패키지의 루트 디렉터리이고 sound, graphic, play는 서브 디렉터리이다.

(※ __init__.py 파일은 조금 특이한 용도로 사용되는데, 이것에 대해서는 뒤에서 자세하게 다룰 것이다.)

간단한 파이썬 프로그램이 아니라면 이렇게 패키지 구조로 파이썬 프로그램을 만드는 것이 공동 작업이나 유지 보수 등 여러 면에서 유리하다. 또한 패키지 구조로 모듈을 만들면 다른 모듈과 이름이 겹치더라도 더 안전하게 사용할 수 있다.

패키지 만들기

이제 위의 예와 비슷한 game 패키지를 직접 만들어 보며 패키지에 대해서 알아보자.

패키지 기본 구성 요소 준비하기

1. C:/Python이라는 디렉터리 밑에 game 및 기타 서브 디렉터리들을 생성하고 .py 파일들을 다음과

같이 만들어 보자(만약 `C:/Python`이라는 디렉터리가 없다면 먼저 생성하고 진행하자).

```
C:/Python/game/_init_.py
```

```
C:/Python/game/sound/_init_.py
```

```
C:/Python/game/sound/echo.py
```

```
C:/Python/game/graphic/_init_.py
```

```
C:/Python/game/graphic/render.py
```

2. 각 디렉터리에 `_init_.py` 파일을 만들어 놓기만 하고 내용은 일단 비워 둔다.

3. `echo.py` 파일은 다음과 같이 만든다.

```
# echo.py
```

```
def echo_test():
```

```
    print ("echo")
```

4. `render.py` 파일은 다음과 같이 만든다.

```
# render.py
```

```
def render_test():
```

```
    print ("render")
```

5. 다음 예제들을 수행하기 전에 우리가 만든 `game` 패키지를 참조할 수 있도록 다음과 같이 도스 창에서 `set` 명령을 이용하여 `PYTHONPATH` 환경 변수에 `C:/Python` 디렉터를 추가한다. 그리고 파이썬 인터프리터(Interactive shell)를 실행하자.

```
C:\> set PYTHONPATH=C:/Python
```

```
C:\> python
```

```
Python 3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:54:25) [MSC v.1900 64 bit (AM...
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

여기까지 준비가 되었다면 다음을 따라 해보자.

패키지 안의 함수 실행하기

자, 이제 패키지를 이용하여 `echo.py` 파일의 `echo_test` 함수를 실행해 보자. 패키지 안의 함수를 실행하는 방법은 다음과 같이 3가지가 있다. 아래 예제들은 `import` 예제들이므로 하나의 예제를 실행하고 나서 다음 예제를 실행할 때에는 반드시 인터프리터를 종료하고 다시 실행해야 한다. 인터프리터를 다시 시작하지 않을 경우 이전에 `import`했던 것들이 메모리에 남아 있게 되어 엉뚱한 결과가 나올 수 있다(윈도우의 경우 인터프리터 종료는 `Ctrl+Z`).

첫 번째는 `echo` 모듈을 `import`하여 실행하는 방법으로, 다음과 같이 실행한다.

```
>>> import game.sound.echo
>>> game.sound.echo.echo_test()
echo
```

두 번째는 echo 모듈이 있는 디렉터리까지를 from ... import하여 실행하는 방법이다.

```
>>> from game.sound import echo
>>> echo.echo_test()
echo
```

세 번째는 echo 모듈의 echo_test 함수를 직접 import하여 실행하는 방법이다.

```
>>> from game.sound.echo import echo_test
>>> echo_test()
echo
```

하지만 다음과 같이 echo_test 함수를 사용하는 것은 불가능하다.

```
>>> import game
>>> game.sound.echo.echo_test()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'module' object has no attribute 'sound'
```

import game을 수행하면 game 디렉터리의 모듈 또는 game 디렉터리의 `__init__.py`에 정의된 것들만 참조할 수 있다.

또 다음처럼 echo_test 함수를 사용하는 것도 불가능하다.

```
>>> import game.sound.echo.echo_test
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: No module named echo_test
```

도트 연산자(.)를 사용해서 import a.b.c처럼 import할 때 가장 마지막 항목인 c는 반드시 모듈 또는 패키지여야만 한다.

`__init__.py`의 용도

`__init__.py` 파일은 해당 디렉터리가 패키지의 일부임을 알려주는 역할을 한다. 만약 game, sound, graphic등 패키지에 포함된 디렉터리에 `__init__.py` 파일이 없다면 패키지로 인식되지 않는다.

(※ python3.3 버전부터는 `__init__.py` 파일 없이도 패키지로 인식이 된다([PEP 420](#)). 하지만 하위 버전 호환을 위해 `__init__.py`파일을 생성하는 것이 안전한 방법이다.)

실무에서는 명시적, 관례적으로 `__init__` 둠.

+ `__init__.py` 용도
패키지 초기화 코드
외부에 노출할 항목 정리
해당 디렉터리가 패키지 일부임을 알려줌

시험 삼아 sound 디렉터리의 **init.py**를 제거하고 다음을 수행해 보자.

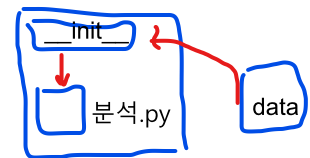
```
>>> import game.sound.echo
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

ImportError: No module named sound.echo

sound 디렉터리에 **__init__.py** 파일이 없어서 임포트 오류(ImportError)가 발생하게 된다.



data가 매일 들어와 해당 모듈에서 분석해야 한다.

data를 __init__에서 초기화하는 역할
예. chap05에 game.sound.__init__.py
해당 환경을 체크하고 초기화
__init__에서 해당 모듈 실행 전 작업을 할 수o

all의 용도

다음은 따라 해보자.

```
>>> from game.sound import *
```

```
>>> echo.echo_test()
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

NameError: name 'echo' is not defined

그런데 뭔가 이상하지 않은가? 분명 game.sound 패키지에서 모든 것(*)을 import하였으므로 echo 모듈을 사용할 수 있어야 할 것 같은데 echo라는 이름이 정의되지 않았다는 이름 오류(NameError)가 발생했다.

이렇게 특정 디렉터리의 모듈을 *를 이용하여 import할 때에는 다음과 같이 해당 디렉터리의 **__init__.py** 파일에 **__all__**이라는 변수를 설정하고 import할 수 있는 모듈을 정의해 주어야 한다.

```
# C:/Python/game/sound/__init__.py
__all__ = ['echo']
```

여기서 **__all__**이 의미하는 것은 sound 디렉터리에서 * 기호를 이용하여 import할 경우 이곳에 정의된 echo 모듈만 import된다는 의미이다.

(※ 착각하기 쉬운데 **from game.sound.echo import ***는 **__all__**과 상관없이 무조건 import된다. 이렇게 **__all__**과 상관없이 무조건 import되는 경우는 **from a.b.c import ***에서 from의 마지막 항목인 c가 모듈인 경우이다.)

위와 같이 **__init__.py** 파일을 변경한 후 위 예제를 수행하면 원하던 결과가 출력되는 것을 확인할 수 있다.

```
>>> from game.sound import *
```

```
>>> echo.echo_test()
```

echo

relative 패키지

만약 graphic 디렉터리의 render.py 모듈이 sound 디렉터리의 echo.py 모듈을 사용하고 싶다면 어떻게 해야 할까? 다음과 같이 render.py를 수정하면 가능하다.

```
# render.py
from game.sound.echo import echo_test
def render_test():
    print ("render")
    echo_test()
```

import 방식

절대(권장) root부터 game.sound.echo
상대 graphic 기준으로 sound import

from game.sound.echo import echo_test라는 문장을 추가하여 echo_test() 함수를 사용할 수 있도록 수정했다.

이렇게 수정한 후 다음과 같이 수행해 보자.

```
>>> from game.graphic.render import render_test
>>> render_test()
render
echo
```

이상 없이 잘 수행된다.

위 예제처럼 from game.sound.echo import echo_test와 같이 전체 경로를 이용하여 import할 수도 있지만 다음과 같이 relative하게 import하는 것도 가능하다.

(※ 이 기능은 Python 2.5부터 지원되기 시작하였다.)

```
# render.py
from ..sound.echo import echo_test

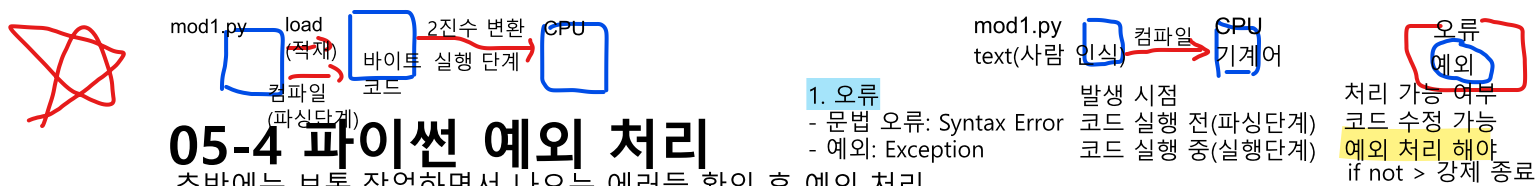
def render_test():
    print ("render")
    echo_test()
```

from game.sound.echo import echo_test가 from ..sound.echo import echo_test로 변경되었다. 여기서 ..은 부모 디렉터리를 의미한다. graphic과 sound 디렉터리는 동일한 깊이(depth)이므로 부모 디렉터리(..)를 이용하여 위와 같은 import가 가능한 것이다.

relative한 접근자에는 다음과 같은 것들이 있다.

- .. – 부모 디렉터리
- . – 현재 디렉터리

..과 같은 relative한 접근자는 render.py와 같이 모듈 안에서만 사용해야 한다. 파이썬 인터프리터에서 relative한 접근자를 사용하면 "SystemError: cannot perform relative import"와 같은 오류가 발생한다.



05-4 파이썬 예외 처리

초반에는 보통 작업하면서 나오는 에러들 확인 후 예외 처리
경험 쌓이면 예측해서 예외 처리

프로그램을 만들다 보면 수없이 많은 오류를 만나게 된다. 물론 오류가 발생하는 이유는 프로그램이 잘못 동작되는 것을 막기 위한 파이썬의 배려이다. 하지만 ~~때때로 이러한 오류를 무시하고 싶을 때도 있고 별도로 처리하고 싶을 때도 있다.~~ 이에 파이썬은 **try, except**를 이용해서 오류를 처리할 수 있게 해준다.

오류는 어떤 때 발생하는가?

막을 수 있는 오류: 오타, 휴먼 에러(수정 가능) (실행 전에 잡을 수 o)

막을 수 없는 오류: 예외 상황 => 실행 중 중지됨.
발생할 수도 아닐 수도 있음.(대비를 하고 있어야 함.)

오류를 처리하는 방법을 알기 전에 어떤 상황에서 오류가 발생하는지 한번 알아보자. 오타를 쳤을 때 발생하는 구문 오류 같은 것이 아닌 실제 프로그램에서 자주 발생하는 오류를 중심으로 살펴본다.

먼저 디렉터리 안에 없는 파일을 열려고 시도했을 때 발생하는 오류이다.

```
>>> f = open("나없는파일", 'r') 읽기 모드(+추가 쓰기 모드)에서 파일 없으면 error
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: '나없는파일'
```

위의 예에서 볼 수 있듯이 없는 파일을 열려고 시도하면 "FileNotFoundError"라는 이름의 오류가 발생하게 된다.

(※ python 2.7 버전에서는 "FileNotFoundError"가 아닌 "IOError"라는 이름의 오류가 발생한다.)

이번에는 0으로 다른 숫자를 나누는 경우를 생각해 보자.

예. 지하철 문 오류

- 문이 안 열림: 예외(큰 지장x) -> 안 멈추고 안내만
- 문이 안 닫힘: 오류(큰 영향..) -> 멈춰야.

```
>>> 4 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

중지 -> 막아야

4를 0으로 나누려니까 "ZeroDivisionError"라는 이름의 오류가 발생한다.

마지막으로 한 가지 예만 더 들어 보자. 다음 오류는 정말 빈번하게 일어난다.

```
>>> a = [1,2,3]
>>> a[4]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

중지 -> 막아야

a는 [1, 2, 3]이라는 리스트인데 a[4]는 a 리스트에서 얻을 수 없는 값이다. 따라서 "IndexError"가 발생하게 된다. 파이썬은 이런 오류가 발생하면 프로그램을 중단하고 오류메시지를 보여 준다.

오류 예외 처리 기법

2. 예외 처리 목적

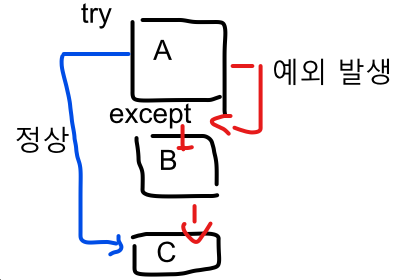
- 프로그램 실행이 강제 종료 x -> 일부 작업 실패해도 전체 동작
- 문제 상황을 깔끔하게 정리 -> 뒷정리(파일, 네트워크의 close())
- 사용자에게 이해 가능한 메시지 출력 -> 에러 설명 메시지 출력(왜 오류?)
- Logging: 오류 기록(+정상, 오류 실행 단계, 상태 기록) -> 복구

자, 이제 유연한 프로그래밍을 위한 오류 처리 기법에 대해서 살펴보자.

try, except문

try -> 예외 발생 -> 원인 -> object 생성 -> exception 블록

다음은 오류 처리를 위한 try, except문의 기본 구조이다.



try:

... 예외 발생할 수 있는 코드를 try block에 위치(위험한 코드)

except [발생 오류 [as 오류 메시지 변수]]:

... 예외 발생 시 except block 코드 실행

아래 코드는 예외 발생하든 정상 작동이든 실행되어야 한다.

try 블록 수행 중 오류가 발생하면 except 블록이 수행된다. 하지만 try 블록에서 오류가 발생하지 않는다면 except 블록은 수행되지 않는다.

except 구문을 자세히 살펴보자.

except [발생 오류 [as 오류 메시지 변수]]:

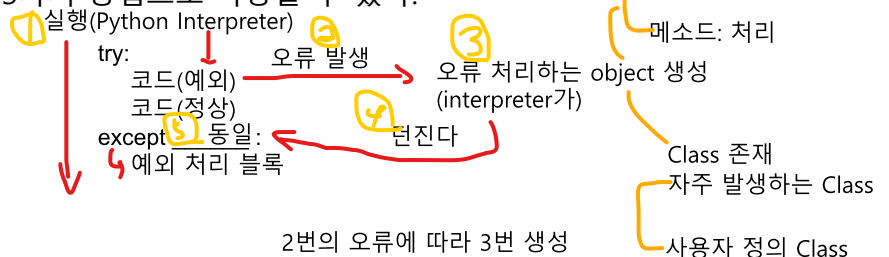
위 구문을 보면 [] 기호를 사용하는데, 이 기호는 괄호 안의 내용을 생략할 수 있다는 관례적인 표기법이다. 즉, except 구문은 다음처럼 3가지 방법으로 사용할 수 있다.

1. try, except만 쓰는 방법

try: 하나의 try 블록 안에
... 최소의 예외 코드가 들어가도록

except:

...



이 경우는 오류 종류에 상관없이 오류가 발생하기만 하면 except 블록을 수행한다.

2. 발생 오류만 포함한 except문

try:

...

except 발생 오류:

...

(BaseException은 직접 호출x)

BaseException	이유
- SystemExit	sys.exit() 호출 시
- KeyboardInterrupt	터미널에서 ctrl + c 누를 때
- GeneratorExit	일반적인 모든 예외의 부모
- Exception	연산 오류 발생 시
- ArithmeticError	연산 오류 발생 시
- LookupError(IndexError(list), KeyError(dict))	객체에 없는 속성에 접근할 때
- AttributeError	정의 안 된 변수 사용할 때
- NameError	잘못된 타입 연산할 때
- TypeError	값이 잘못되었을 때
- ValueError	
- OSError	
- FileNotFoundError	파일 못 찾을 때
- PermissionError	권한이 없을 때
- TimeoutError	
- RuntimeError	보안 관련
- RecursionError	
- ImportError	
- ModuleNotFoundError	

이 경우는 오류가 발생했을 때 except문에 미리 정해 놓은 오류 이름과 일치할 때만 except 블록을 수행한다는 뜻이다.

3. 발생 오류와 오류 메시지 변수까지 포함한 except문

try:

...

except 발생 오류 as 오류 메시지 변수:

...

3. 예외 처리 구조

- 3.1 모든 예외는 "클래스"
- 만들어져 있는
- 만들 예정인

이 경우는 두 번째 경우에서 오류 메시지의 내용까지 알고 싶을 때 사용하는 방법이다.

이 방법의 예를 들어 보면 다음과 같다.

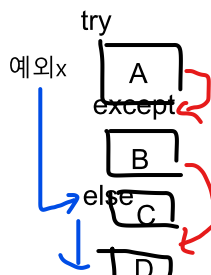
```
try:
    4 / 0
except ZeroDivisionError as e:
    print(e)
```

(※ 파이썬 2.7 버전의 경우에는 위 예제의 `except ZeroDivisionError as e:` 대신 `except ZeroDivisionError, e:`와 같이 사용해야 한다.)

위처럼 4를 0으로 나누려고 하면 `ZeroDivisionError`가 발생하여 `except` 블록이 실행되고 `e`라는 오류 메시지를 다음과 같이 출력한다.

결과값: division by zero

```
except
try .. else
```



예외가 발생하지 않았을 때 처리 코드

`try`문은 `else`절을 지원한다. `else`절은 예외가 발생하지 않은 경우에 실행되며 반드시 `except`절 바로 다음에 위치해야 한다.

(※ `else`절은 `else` 블록과 같은 뜻이다.)

다음 예를 보자.

```
try:
    f = open('foo.txt', 'r')
except FileNotFoundError as e:
    print(str(e))
else:
    data = f.read()
    f.close()
```

만약 `foo.txt`라는 파일이 없다면 `except`절이 수행되고 `foo.txt` 파일이 있다면 `else`절이 수행될 것이다.

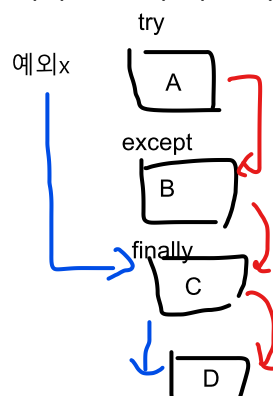
try .. finally

예외가 발생하든 안 하든 항상 실행될 코드(close)
마무리 코드

`try`문에는 `finally`절을 사용할 수 있다. `finally`절은 `try`문 수행 도중 예외 발생 여부에 상관없이 항상 수행된다. 보통 `finally`절은 사용한 리소스를 `close`해야 할 경우에 많이 사용된다.

다음의 예를 보자.

```
f = open('foo.txt', 'w')
try:
    # 무언가를 수행한다.
finally:
    f.close()
```



foo.txt라는 파일을 쓰기 모드로 연 후에 try문이 수행된 후 예외 발생 여부에 상관없이 finally절에서 f.close()로 열린 파일을 닫을 수 있다.

여러개의 오류처리하기

try문 내에서 여러개의 오류를 처리하기 위해서는 다음과 같은 구문을 이용한다.

```
try:
    ...
except 발생 오류1:
    ...
except 발생 오류2:
    ...
```

즉, 다음과 같이 0으로 나누는 오류와 인덱싱 오류를 다음과 같이 처리할 수 있다.

```
try:
    a = [1,2]
    print(a[3])
    4/0
except ZeroDivisionError:
    print("0으로 나눌 수 없습니다.")
except IndexError:
    print("인덱싱 할 수 없습니다.")
```

try 블록에서 IndexError가 먼저 나서 그 후 try는 실행x

IndexError Object
생성

a는 2개의 요소값을 가지고 있기 때문에 a[3]은 IndexError를 발생시키므로 "인덱싱 할 수 없습니다."라는 문자열이 출력될 것이다. 인덱싱 오류가 먼저 발생했으므로 4/0으로 발생되는 ZeroDivisionError는 발생하지 않았다.

이전에 알아보았던 것과 마찬가지로 오류메시지도 다음과 같이 가져올 수 있다.

```
Python Interpreter
try:
    a = [1,2]
    print(a[3])
    4/0
except ZeroDivisionError as e:
    print(e)
except IndexError as e:
    print(e)
```

변수: 왜?(문자열)
메소드
오류 발생 -> IndexError object 생성

실행하면 "list index out of range"라는 오류 메시지가 출력될 것이다.

(비권장) 다음과 같이 ZeroDivisionError와 IndexError를 함께 처리할 수도 있다.

```
try:
    a = [1,2]
    print(a[3])
    4/0
except (ZeroDivisionError, IndexError) as e:
    print(e)
```

2개 이상의 오류를 동시에 처리하기 위해서는 위와같이 괄호를 이용하여 함께 묶어주어 처리하면 된다.

✕ 오류 회피하기

프로그래밍을 하다 보면 특정 오류가 발생할 경우 그냥 통과시켜야 할 때가 있을 수 있다. 다음의 예를 보자.

try:

```
f = open("나없는파일", 'r')
```

except FileNotFoundError:

~~pass~~

try문 내에서 FileNotFoundError가 발생할 경우 pass를 사용하여 오류를 그냥 회피하도록 한 예제이다.

오류 일부러 발생시키기

(강제로)

반드시 오버라이딩 처리하겠다.
(python에서는 해당 명령어가 x. 수동 처리)

직접 실행을 막겠다.

이상하게 들리겠지만 프로그래밍을 하다 보면 종종 오류를 일부러 발생시켜야 할 경우도 생긴다. 파이션은 raise라는 명령어를 이용해 오류를 강제로 발생시킬 수 있다.

예를 들어 Bird라는 클래스를 상속받는 자식 클래스는 반드시 fly라는 함수를 구현하도록 만들고 싶은 경우(강제로 그렇게 하고 싶은 경우)가 있을 수 있다. 다음 예를 보자.

```
class Bird:
```

```
    def fly(self):
```

```
        raise NotImplementedError
```

위 예제는 Bird 클래스를 상속받는 자식 클래스는 반드시 fly라는 함수를 구현해야 한다는 의지를 보여준다. 만약 자식 클래스가 fly 함수를 구현하지 않은 상태로 fly 함수를 호출한다면 어떻게 될까?

(※ NotImplementedError는 파이썬 내장 오류로, 꼭 작성해야 하는 부분이 구현되지 않았을 경우 일부러 오류를 발생시키고자 사용한다.)

```
class Eagle(Bird):
```

```
    pass
```

```
eagle = Eagle()
```

```
eagle.fly()
```

Eagle 클래스는 Bird 클래스를 상속받는다. 그런데 Eagle 클래스에서 fly 함수를 구현하지 않았기 때문에 Bird 클래스의 fly 함수가 호출된다. 그리고 raise문에 의해 다음과 같은 NotImplementedError가 발생할 것이다.

(※ 상속받는 클래스에서 함수를 재구현하는 것을 메서드 오버라이딩이라고 부른다.)

Traceback (most recent call last):

```
File "...", line 33, in <module>
    eagle.fly()
File "...", line 26, in fly
    raise NotImplementedError
NotImplementedError
```

NotImplementedError가 발생되지 않게 하려면 다음과 같이 Eagle 클래스에 fly 함수를 반드시 구현해야 한다.

```
class Eagle(Bird):
    def fly(self):
        print("very fast")

eagle = Eagle()
eagle.fly()
```

위 예처럼 fly 함수를 구현한 후 프로그램을 실행하면 오류 없이 다음과 같은 문장이 출력된다.

```
very fast
```

오류 만들기

프로그램 수행 도중 특수한 경우에만 예외처리를 하기 위해서 종종 오류를 만들어서 사용하게 된다.

직접 오류를 만들어 보자. 오류는 다음과 같이 파이썬 내장 클래스인 `Exception` 클래스를 상속하여 만들 수 있다.

```
class MyError(Exception):
    pass
```

그리고 별명을 출력해 주는 함수를 다음과 같이 작성해 보자.

```
def say_nick(nick):
    if nick == '바보':
        raise MyError()
    print(nick)
```

그리고 다음과 같이 say_nick 함수를 호출 해 보자.

```
say_nick("천사")
say_nick("바보")
```

실행 해 보면 다음과 같이 "천사"가 한번 출력된 후 `MyError`가 발생하는 것을 알 수 있다.

```
천사
```

```
Traceback (most recent call last):
  File "...", line 11, in <module>
    say_nick("바보")
  File "...", line 7, in say_nick
    raise MyError()
```

```
__main__.MyError
```

이번에는 `MyError`가 발생할 경우 예외처리기법을 이용하여 예외처리를 해 보도록 하자.

```
try:
    say_nick("천사")
    say_nick("바보")
except MyError:
    print("허용되지 않는 별명입니다.")
```

실행 하면 다음과 같이 출력된다.

```
천사
허용되지 않는 별명입니다.
```

만약 오류메시지를 이용하고 싶다면 다음처럼 예외처리를 해야 할 것이다.

```
try:
    say_nick("천사")
    say_nick("바보")
except MyError as e:
    print(e)
```

하지만 실행 해 보면 `print(e)`로 출력한 오류메시지가 아무것도 출력되지 않는것을 확인 할 수 있다. 오류 메시지를 출력했을 때 오류 메시지가 보이게 하기 위해서는 오류 클래스에 다음과 같은 `__str__` 메서드를 구현해야 한다. `__str__` 메서드는 `print(e)` 처럼 오류메시지를 print문으로 출력할 경우에 호출되는 메서드이다.

```
class MyError(Exception):
    def __str__(self):
        return "허용되지 않는 별명입니다."
```

다시 실행해 보면 "허용되지 않는 별명입니다."라는 오류메시지가 출력되는 것을 확인할 수 있다. 만약 에러 발생시점에 오류메시지를 전달하고 싶다면 다음과 같이 수정해야 한다.

```
class MyError(Exception):
    def __init__(self, msg):
        self.msg = msg

    def __str__(self):
        return self.msg

def say_nick(nick):
    if nick == '바보':
        raise MyError("허용되지 않는 별명입니다.")
    print(nick)

try:
    say_nick("천사")
```

```
say_nick("바보")  
except MyError as e:  
    print(e)
```

`raise MyError("허용되지 않는 별명입니다.")`처럼 오류 발생시점에 메시지를 전달할 수 있다.